

SENA — ADSO 2026

INFORME TÉCNICO

Investigación e Implementación de API Externa

API de Supabase — Base de Datos PostgreSQL en la Nube

Integración en Texticode — Sistema de Gestión Textil

Aprendiz

Kevin Stevan Wagner

Kevin Andres Castro

Andres Felipe Lancheros

Camilo Andres Tibambre

Instructor

Ing. Edwin Marín

Fecha: Mayo 2026

Proyecto: Texticode — ADSO 2026

★ Implementación real verificada con evidencias de prueba ★

1. API IMPLEMENTADA: SUPABASE — BASE DE DATOS POSTGRESQL EN LA NUBE

1.1 Justificación de la elección

La API de Supabase es una API externa que provee acceso a una base de datos PostgreSQL gestionada en la nube mediante una interfaz de conexión estándar. El proyecto Texticode la implementó usando el driver pg (node-postgres) con un Pool de conexiones apuntando al Session Pooler de Supabase, alojado en us-west-1 (North California) con Project ID oizxmymoagvasyculdma.

A diferencia de la conexión MySQL local anterior, Supabase permite que los 4 miembros del equipo ADSO (stevanwagner1, andreslanck, camilotibambre y un cuarto integrante) accedan al mismo servidor de base de datos desde cualquier equipo, sin necesidad de tener una instancia local corriendo.

Necesidad en Texticode	Solución implementada con Supabase
La BD MySQL local solo funcionaba en el equipo del desarrollador — sin acceso remoto ni trabajo colaborativo	Supabase provee PostgreSQL gestionado en la nube (us-west-1), accesible desde cualquier red con la DATABASE_URL del Session Pooler
No había forma de acceder a los datos de producción desde múltiples equipos del equipo ADSO	Los 4 miembros del equipo (stevanwagner1, andreslanck, camilotibambre) acceden al mismo proyecto Supabase como Administrators
La conexión MySQL requería que el servidor Node estuviera en el mismo equipo que la BD	El backend Node.js conecta a Supabase vía pg.Pool con connection string: aws-0-us-west-1.pooler.supabase.com:6543
No había monitoreo de peticiones ni	Supabase Dashboard muestra en tiempo real Database

métricas de uso de la BD

Requests, Auth Requests y métricas de CPU/RAM/Disk del servidor

1.2 Problema que resuelve la API

Texticode operaba con MySQL local que solo funcionaba en el equipo donde corría el servidor Node.js. Esto generaba tres problemas concretos:

- Base de datos no portable: cada miembro del equipo tenía que importar el schema MySQL manualmente y mantener su propia instancia local sincronizada con los demás.
- Sin monitoreo ni métricas: no había forma de ver cuántas peticiones llegaban a la BD, ni el estado del servidor en tiempo real. Supabase provee Dashboard con Database Requests, CPU, RAM y Disk en tiempo real.
- Autenticación manual compleja: el sistema de tokens de recuperación de contraseña y JWT debía gestionarse completamente en Node. Con Supabase, la infraestructura de base de datos está disponible 24/7 sin mantenimiento.

2. INVESTIGACIÓN TÉCNICA DE LA API

2.1 Descripción técnica

Tipo: API REST externa | Proveedor: Supabase | BD: PostgreSQL | Región: us-west-1 (North California) | Conexión: pg.Pool + Session Pooler | Puerto: 6543

2.2 Métodos HTTP y endpoints probados

Método	Endpoint	Uso en Texticode	Respuesta
GET	/api/usuarios	Lista todos los usuarios con su rol (administrador, operario, cliente)	200 OK + JSON array — 11 registros
GET	/api/ordenes	Lista todas las órdenes de producción con cliente y material JOIN	200 OK + JSON array — 20 registros
GET	/api/materiales	Lista todos los materiales con nombre del cliente asignado	200 OK + JSON array — 11 registros
GET	/api/eficiencia/operarios	Lista operarios con métricas: prendas/día, retrasos, rendimiento	200 OK + {ok, total:5, data:[...]}
GET	/api/eficiencia/operarios/28	Perfil completo de Erika Parada con todas sus órdenes (Path param)	200 OK + {ok, data:{...ordenes_detalle}}
GET	/api/eficiencia/operarios?rendimiento=Alto	Filtra operarios por nivel de rendimiento (Query param)	200 OK + {total:0, data:[]}
POST	/api/ordenes	Crea nueva orden — devolvió Id_Orden: 133	201 Created + {mensaje, Id_Orden:133}

PUT	/api/ordenes/133	Actualiza estado y datos de la orden Id_Orden 133	200 OK + {mensaje: 'Orden actualizada'}
DELETE	/api/ordenes/133	Elimina la orden de prueba Id_Orden 133	200 OK + {mensaje: 'Orden eliminada'}

2.3 Autenticación y configuración de conexión

La conexión a Supabase desde el backend Node.js usa el driver nativo pg (node-postgres) con un Pool de conexiones. La autenticación se realiza a través de la DATABASE_URL que incluye el usuario, contraseña del proyecto, host del Session Pooler y nombre de la base de datos. SSL está habilitado con rejectUnauthorized: false tal como lo requiere Supabase.

Mecanismo	Descripción / Valor en Texticode
DATABASE_URL	Connection string completa almacenada en .env del backend. Nunca expuesta al frontend ni al repositorio.
Session Pooler	aws-0-us-west-1.pooler.supabase.com:6543 — compatible con redes IPv4. Gestiona el pool de conexiones automáticamente.
SSL	ssl: { rejectUnauthorized: false } — requerido por Supabase para conexiones seguras desde pg.Pool.
Project ID	oizxmymoagvasyculdma — identifica el proyecto Texticode dentro de la plataforma Supabase.
Acceso de equipo	4 miembros del equipo ADSO con roles Owner y Administrator en el proyecto Supabase.

```
// texticode-api/db.js — conexión real a Supabase PostgreSQL
import pg from 'pg'
import dotenv from 'dotenv'
dotenv.config()

const { Pool } = pg

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: { rejectUnauthorized: false } // requerido por Supabase
})

// Verificar conexión al iniciar el servidor
try {
  const client = await pool.connect()
  console.log('✓ Conectado a Supabase PostgreSQL')
  client.release()
} catch (err) {
  console.error('✗ Error conectando a Supabase:', err.message)
```

```
process.exit(1)
}

export default pool

// texticode-api/.env — NUNCA subir al repositorio
DATABASE_URL=postgresql://postgres.oizxmymoagvasyculdma:[PASSWORD]@aws-0-us-west-1.pooler.supabase.com:6543/postgres
```

2.4 Parámetros disponibles

Tipo	Parámetro	Ejemplo real	Comportamiento
Path param	:id en /operarios/:id	/api/eficiencia/operarios/28	Devuelve perfil completo de Erika Parada (Id_Usuario: 28) con ordenes_detalle
Path param	:id en /ordenes/:id	/api/ordenes/133	Devuelve, actualiza o elimina la orden con Id_Orden 133
Query param	?rendimiento=Alto Medio Bajo	?rendimiento=Alto → total: 0	Filtra operarios por nivel de rendimiento calculado. Rechaza valores inválidos con 400
Query param	?estado=Completada En Proceso Pausado	?estado=En+Proceso	Filtra operarios que tengan órdenes en ese estado específico
Query param	?limite=N	?limite=3	Limita el número de operarios devueltos. Rechaza valores no numéricos con 400
Path param	:idCliente en /cliente/:id	/api/ordenes/cliente/33	Filtra órdenes del cliente Patprimo S.A.S. (Id_Usuario: 33)

3. IMPLEMENTACIÓN TÉCNICA REAL EN TEXTICODE

3.1 Estructura del proyecto con la API implementada

```
TEXTICODE-VUE/      ← Raíz del workspace
├── texticode-api/   ← Backend Node.js + Express
│   ├── routes/
│   │   ├── auth.route.js    ← Login, recuperación contraseña (db.query)
│   │   ├── usuarios.js      ← CRUD usuarios con bcrypt (pool.query)
│   │   ├── ordenes.js       ← CRUD órdenes de producción (pool.query)
│   │   ├── materiales.js    ← CRUD inventario (pool.query)
│   │   ├── comprobantes.js  ← Gestión comprobantes (pool.query)
│   │   ├── eficiencia.js    ← Métricas operarios (pool.query SQL)
│   │   └── carga_trabajo.js ← Análisis carga (pool.query)
```

	googleCalendar.js	← OAuth Google Calendar
	notificaciones.js	← SendGrid email service
	roles.js	← CRUD roles (pool.query)
	orden_material.js	← Relación orden-material (pool.query)
	usuario_orden.js	← Asignación operarios (pool.query)
	practica.routes.js	← Path/Query params de práctica
	db.js	← ★ pg.Pool conectado a Supabase PostgreSQL
	.env	← ★ DATABASE_URL=Session Pooler Supabase
	index.js	← Entry point — 13 routers registrados
	texticode-vue/	← Frontend Vue 3 + Vite
	src/views/admin/	← GestionUsuarios, Produccion, Inventario...

Archivo	Rol en la integración con Supabase
db.js	Inicializa pg.Pool con DATABASE_URL del Session Pooler de Supabase. Incluye ssl: { rejectUnauthorized: false } requerido por Supabase. Verifica la conexión al arrancar el servidor.
.env (backend)	Almacena DATABASE_URL=postgresql://postgres.oizxmymoagvasyculdma:[PASSWORD]@aws-0-us-west-1.pooler.supabase.com:6543/postgres. Nunca sube al repositorio.
routes/ordenes.js	Usa pool.query() con SQL parametrizado para CRUD completo. JOIN con usuario y material para devolver Cliente y NombreMaterial en cada orden.
routes/usuarios.js	CRUD con bcrypt para hash de contraseñas. Usa pool.query() para SELECT, INSERT, UPDATE con parámetros \$1, \$2... contra Supabase PostgreSQL.
routes/eficiencia.js	SQL complejo con COUNT, CASE, ROUND, CURDATE() sobre las tablas usuario, rol y orden_produccion de Supabase. Calcula métricas en tiempo real.
routes/auth.route.js	Login con JWT, recuperación de contraseña con tokens_recuperacion y SendGrid. Todas las queries van contra Supabase PostgreSQL vía db.query().

3.2 Código del endpoint con Path param — Perfil de operario

🔍 FUNCIONALIDAD 1: Perfil completo de operario por ID (Path param)

Cuando se consulta GET /api/eficiencia/operarios/28, el sistema ejecuta una query SQL compleja contra Supabase PostgreSQL con JOIN entre usuario, rol y orden_produccion, calculando métricas en tiempo real. Para Erika Parada devolvió: 3.0 prendas/día, 130 unidades producidas, 2 órdenes en retraso, rendimiento Bajo.

```
// routes/eficiencia.js — Path param :id
import pool from './db.js' // ← pool conectado a Supabase

router.get('/operarios/:id', async (req, res) => {
  const { id } = req.params
  if (isNaN(id))
```

```

return res.status(400).json({ ok: false, mensaje: 'El id debe ser un número válido' })

const [rows] = await pool.query(` // ← pool.query va a Supabase PostgreSQL
SELECT u."Id_Usuario", u."Nombre_Completo", u."Nombre_Usuario",
COUNT(CASE WHEN op."Estado" = 'Completada' THEN 1 END) AS ordenes_completadas,
COALESCE(SUM(op."Unidades_Realizadas"), 0) AS total_unidades_producidas,
ROUND(COUNT(CASE WHEN op."Estado" = 'Completada' THEN 1 END) * 100.0
/ NULLIF(COUNT(op."Id_Orden"), 0), 1) AS prendas_por_dia,
CASE
WHEN COUNT(CASE WHEN CURRENT_DATE > op."Fecha_Limite"
AND op."Estado" IN ('En Proceso','Pausado') THEN 1 END) > 0 THEN 'Bajo'
WHEN ... >= 10 THEN 'Alto' WHEN ... >= 5 THEN 'Medio' ELSE 'Bajo'
END AS rendimiento
FROM usuario u
INNER JOIN rol r ON r."Id_Rol" = u."Id_Rol" AND r."Nombre_Rol" = 'operario'
LEFT JOIN orden_produccion op ON op."Id_Operario" = u."Id_Usuario"
WHERE u."Id_Usuario" = $1 AND u."Estado" = 'activo'
GROUP BY u."Id_Usuario",
[id])

if (!rows.length)
return res.status(404).json({ ok: false, mensaje: 'No se encontró operario con id ${id}' })
res.json({ ok: true, data: { ...rows[0], ordenes_detalle: ordenes } })
})

```

3.3 Código del endpoint con Query params — Lista filtrada

FUNCIONALIDAD 2: Lista de operarios con filtros (Query params)

Cuando se consulta GET /api/eficiencia/operarios?rendimiento=Alto, el sistema obtiene todos los operarios desde Supabase PostgreSQL y aplica el filtro en memoria. La consulta con rendimiento=Alto devolvió total: 0 porque ningún operario activo alcanza el umbral de ≥ 10 prendas/día actualmente.

```

// routes/eficiencia.js — Query params
router.get('/operarios', async (req, res) => {
const { rendimiento, estado, limite } = req.query

// pool.query ejecuta el SQL contra Supabase PostgreSQL
const { rows } = await pool.query(`
SELECT u."Id_Usuario", u."Nombre_Completo", ... métricas calculadas ...
FROM usuario u
INNER JOIN rol r ON r."Nombre_Rol" = 'operario'
LEFT JOIN orden_produccion op ON op."Id_Operario" = u."Id_Usuario"
WHERE u."Estado" = 'activo'
GROUP BY u."Id_Usuario"
ORDER BY prendas_por_dia DESC`)

```

```

let resultado = rows

// Filtro por rendimiento (Query param)
if (rendimiento) {
  const validos = ['Alto', 'Medio', 'Bajo']
  if (!validos.includes(rendimiento))
    return res.status(400).json({ ok: false, mensaje: 'rendimiento inválido' })
  resultado = resultado.filter(r => r.rendimiento === rendimiento)
}

if (limite) resultado = resultado.slice(0, parseInt(limite))

res.json({ ok: true, total: resultado.length,
  filtros_aplicados: { rendimiento, estado, limite }, data: resultado })
})

```

4. SEGURIDAD Y BUENAS PRÁCTICAS

4.1 Protección de credenciales

La DATABASE_URL contiene la contraseña del proyecto Supabase y nunca debe exponerse. Se almacena exclusivamente en el archivo .env del backend, que está en .gitignore. El frontend Vue no tiene acceso a esta variable — solo el backend Node.js la usa internamente para crear el Pool de conexiones.

```

# texticode-api/.env — archivo local, NUNCA subir al repositorio
DATABASE_URL=postgresql://postgres.oizxmymoagvasyculdma:[PASSWORD]@aws-0-us-west-1.pooler.supabase.com:6543/postgres
JWT_SECRET=cambia_este_secreto_seguro
SENDGRID_API_KEY=SG.xxxxxxxxxxxx
API_KEY_EFICIENCIA=texticode-2026

# texticode-api/.gitignore
.env
node_modules/

```

4.2 Riesgos de seguridad identificados

Riesgo	Mitigación aplicada en Texticode
Exposición de DATABASE_URL con contraseña	Almacenada exclusivamente en .env del backend. Archivo en .gitignore — nunca sube al repositorio. El frontend no tiene acceso a esta variable.
Pérdida de conexión por IPv4/IPv6	Se usa Session Pooler (puerto 6543) en lugar de conexión directa, compatible con redes IPv4 colombianas. ssl: rejectUnauthorized: false garantiza la conexión segura.

Inyección SQL en queries parametrizadas	Todas las queries usan placeholders \$1, \$2... con pg.Pool. Ningún valor del usuario se concatena directamente al SQL.
Acceso no autorizado a datos sensibles	Validación de parámetros en cada route (isNaN, includes, campos obligatorios). Respuestas con mensajes controlados sin exponer detalles internos.
Saturación del pool de conexiones (plan free: 200 max)	Connection pool size configurado en 15 conexiones por usuario. Max client connections: 200 (límite del plan Nano de Supabase).
Pérdida de datos del proyecto	Supabase mantiene el proyecto activo mientras haya actividad. El schema SQL completo está versionado para poder recrear la BD si es necesario.

5. PREGUNTAS DE INVESTIGACIÓN

P1. ¿Qué problema resuelve la API en Texticode?

Texticode operaba con MySQL local — la base de datos solo era accesible desde el equipo donde corría el servidor. Los 4 miembros del equipo ADSO no podían trabajar sobre la misma BD simultáneamente. Supabase resuelve esto proveyendo PostgreSQL en la nube accesible desde cualquier equipo. El schema completo de Texticode (11 tablas: rol, usuario, material, orden_produccion, comprobantes, observacion_operario, orden_material, usuario_orden, google_calendar_tokens, tokens_recuperacion) migró a Supabase manteniendo todos los tipos ENUM, foreign keys y restricciones de integridad, con 11 usuarios y 20 órdenes de producción reales.

P2. Métodos HTTP y datos que retorna

GET /api/usuarios: devuelve array de 11 usuarios con Id_Usuario, Nombre_Completo, Nombre_Usuario, Correo, Telefono, Estado, Fecha_Registro, Rol e Id_Rol — con JOIN entre usuario y rol ejecutado por pool.query contra Supabase. GET /api/ordenes: devuelve 20 órdenes con Cliente y NombreMaterial incluidos por JOIN. GET /api/materiales: devuelve 11 materiales con Nombre_Cliente por LEFT JOIN. POST /api/ordenes: devuelve { mensaje: 'Orden creada', Id_Orden: 133 } con status 201 Created. PUT /api/ordenes/133: devuelve { mensaje: 'Orden actualizada' } con status 200 OK. DELETE /api/ordenes/133: devuelve { mensaje: 'Orden eliminada' } con status 200 OK.

P3. ¿Cómo se autentica?

La autenticación con Supabase se realiza mediante la DATABASE_URL en el Pool de conexiones pg. Esta URL incluye usuario (postgres.oizxmymoagvasyculdma), contraseña del proyecto, host del Session Pooler (aws-0-us-west-1.pooler.supabase.com) y puerto 6543. SSL está habilitado con rejectUnauthorized: false. La verificación de la conexión ocurre al iniciar el servidor — si falla, el proceso termina con process.exit(1).

P4. Ventajas de API externa (Supabase) vs API interna (MySQL local)

Al ser una API externa gestionada, Supabase elimina la necesidad de administrar el servidor de base de datos localmente. Los 4 miembros del equipo acceden al mismo proyecto desde sus propios equipos sin configuración adicional. El Dashboard de Supabase provee monitoreo en tiempo real de peticiones, CPU (2%), RAM (46%) y Disk (14%). La desventaja frente a MySQL local es la dependencia de conectividad a internet y los límites del plan gratuito (Nano: 15 conexiones por pool, 200 max client connections, 500MB almacenamiento).

P5. Riesgos de seguridad

El principal riesgo es la exposición de la DATABASE_URL en el repositorio, lo que daría acceso completo a la base de datos del proyecto. Se mitiga con .env + .gitignore. Un segundo riesgo es la inyección SQL si los parámetros se concatenaran directamente — se mitiga con queries parametrizadas \$1, \$2... en pg. Un tercer riesgo es el agotamiento del pool de conexiones en el plan Nano (200 max) con múltiples usuarios simultáneos — se mitiga configurando Connection pool size en 15.

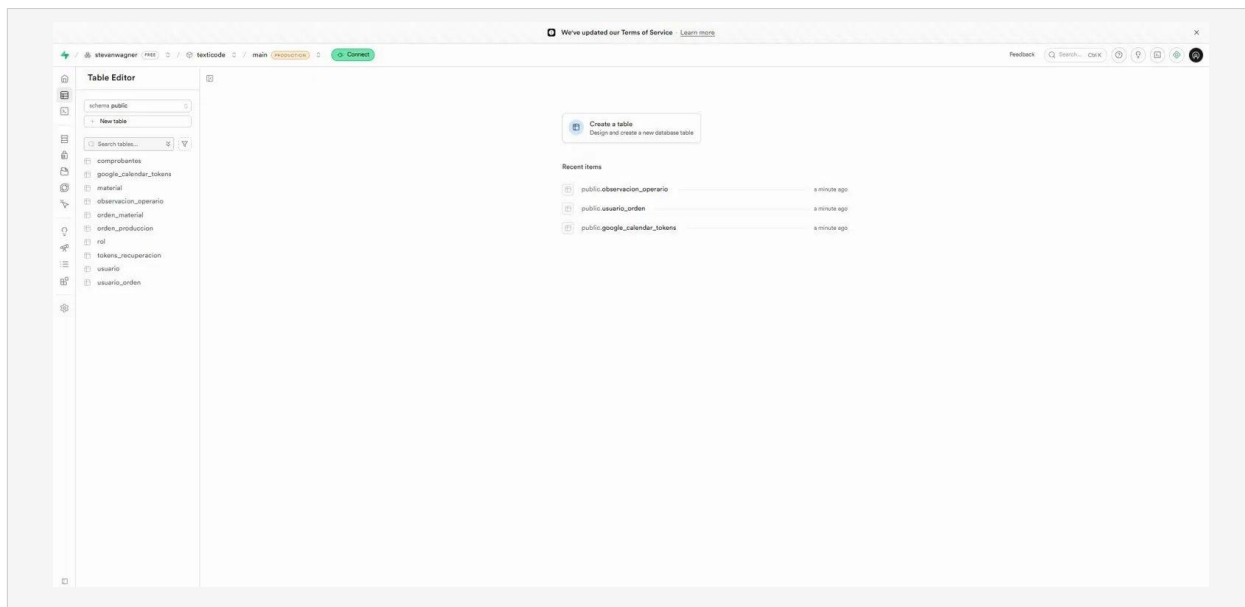
P6. Mejora implementada en Texticode

El schema completo de Texticode migró de MySQL local a Supabase PostgreSQL. El archivo db.js reemplazó la cadena de conexión MySQL por la DATABASE_URL del Session Pooler de Supabase. Todos los routes del backend (13 routers) continuaron usando pool.query() sin modificar ningún SQL ni lógica de negocio. El servidor arranca con ✓ Conectado a Supabase PostgreSQL y todas las operaciones CRUD — GET, POST, PUT, DELETE — funcionan correctamente contra la BD en la nube, verificado con Thunder Client con tiempos de respuesta entre 149ms y 2.12s.

6. EVIDENCIAS DE PRUEBA

6.1 Supabase — Table Editor: 10 tablas del schema Texticode

Vista del Table Editor de Supabase mostrando las 10 tablas del schema public: comprobantes, google_calendar_tokens, material, observacion_operario, orden_material, orden_produccion, rol, tokens_recuperacion, usuario y usuario_orden. Todas las tablas del sistema Texticode fueron migradas exitosamente desde MySQL a PostgreSQL en Supabase.



6.2 Supabase — Tabla usuario con 11 registros reales

Vista de la tabla usuario en Supabase con los 11 usuarios del sistema: 2 administradores (Kevin Wagner, Stevan Wagner), 5 operarios (David Hernández, Erika Parada, Sandra Rincon, Julian Hurtado, Elsa Cardenas, sengrid) y 3 clientes (Patprimo S.A.S., Paola Castro, Andres Fuquene). Las contraseñas están hasheadas con bcrypt (\$2b\$10\$...).

id_usuario	nombre_completo	nombre_usuario	correo	telefono	estado	contraseña	fecha_registro
26	Karin Wagner	Wagner	wagner@gmail.com	+57 311 4000000	activo	\$2b\$10\$BqBtaly7Dggrwckm1Banyes	2026-04-09 22:44:57
27	David Hamadad	Davidcho	stevanwagner@gmail.com	+57 310286674	activo	\$2b\$10\$CnTbndwVvYbawuLrZDQgho	2026-04-09 22:46:49
28	Erika Parada	Erika22	wagnerheidi@gmail.com	+57 320544222	activo	\$2b\$10\$GrgpWwVvRtlywGeh7vRkx	2026-04-09 22:48:36
29	Stevan Wagner	Witeren	stevanwagner@gmail.com	+57 322 442 0001	activo	\$2b\$10\$AhacCzDZNBwFGGShLzCz	2026-04-09 22:49:00
30	Sandra Rincon	Sandrita	stevanwagner@gmail.com	+57 3354567801	activo	\$2b\$10\$TjzPwPwckuFzL5Nu3P	2026-04-09 22:50:51
31	Julian Hurtado	Julii	julian@gmail.com	+57 3253886509	activo	\$2b\$10\$GfWfVvMvL7GmndGZLwvP	2026-04-09 22:58:51
32	Elsa Corderas	Elsa	elsa@gmail.com	+57 3163332223	activo	\$2b\$10\$BvVvVvVvVvVvVvVvVvVv	2026-04-09 23:00:09
33	Pérgimo S.A.S.	Pérgimo	stevanwagner@gmail.com	+57 3205233245	activo	\$2b\$10\$BvVvVvVvVvVvVvVvVvVv	2026-04-09 23:00:45
34	Peña Castro	Peña	stevan@gmail.com	+57 3205088888	activo	\$2b\$10\$BvVvVvVvVvVvVvVvVvVv	2026-04-10 00:19:55
35	Andrés Piqueña	Andrés	stevan@gmail.com	+57 3228647862	activo	\$2b\$10\$BvVvVvVvVvVvVvVvVvVv	2026-04-10 00:20:59
36	segrid	segrid	camilobaneda@gmail.com	+57 000 000 0000	inactivo	\$2b\$10\$BvVvVvVvVvVvVvVvVvVv	2026-05-05 21:40:57

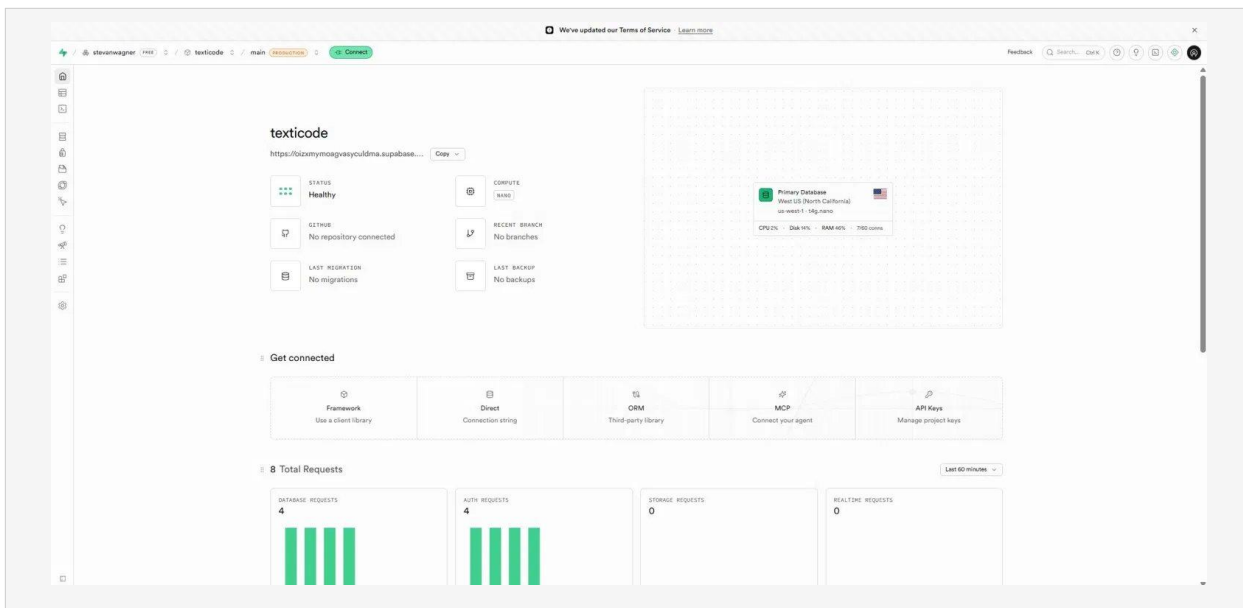
6.3 Supabase — Tabla orden_produccion con 20 órdenes

Vista de la tabla orden_produccion en Supabase con las 20 órdenes de producción reales del sistema: desde Camisas (Id_Orden: 10) hasta GORRAS NEW ERA (Id_Orden: 29). Estados variados: Completada y En Proceso. Productos reales: SUDADERAS HURLINTONG, HODDIES HURLINTONG, CHAQUETAS THE NORTH FACE, UNIFORMES FUTBOL, entre otros.

id_orden	id_cliente	id_material	producto	cantidad	prioridad	fecha_límite	descripción	difusado	estado	unidades_realizadas
10	33	10	Camisas	100	Medio	2026-04-15	Camisas para fiesta fría	Medio	Completada	100
11	33	13	Pijamas	50	Baja	2026-04-23	Pijamas de alta calidad	Medio	Completada	50
12	33	14	Pantalones	2000	Medio	2026-04-17	Pantalones casuales	Medio	Completada	2000
13	34	16	Pantalones	22	Alta	2026-04-15	Pantalones del PSG	Medio	En Proceso	NUL
14	34	17	Uniformes	50	Alta	2026-04-14	Uniformes para niños	Alta	En Proceso	NUL
15	34	16	Pantalones	33	Baja	2026-04-30	Pantalones tipo sudaderas	Medio	En Proceso	NUL
16	35	20	Camisas	10	Baja	2026-05-14	Camisas para mujer y niños	Medio	Completada	10
17	35	20	Medias	40	Medio	2026-04-22	Medias térmicas	Baja	Completada	40
18	35	18	Boaer	500	Alta	2026-04-15	Ropa interior para niños	Baja	Completada	500
19	35	14	camisas oversize	200	Alta	2026-03-07	tail m y y d	Medio	En Proceso	NUL
20	35	9	SUDADERAS HURLINTONG	200	Alta	2026-08-08	TALLAS S, M, L Y XL NEGRAS	Alta	En Proceso	NUL
21	35	10	HODDIES HURLINTONG	1000	Alta	2026-07-05	NEGRAS CON BORDADOS LLAMATIVOS	Alta	En Proceso	NUL
22	35	11	CHAQUETAS THE NORTH FACE	80	Alta	2026-07-05	NEGRA, CON BORDADOS	Alta	Completada	80
23	35	11	camisas sans	50	Medio	2026-07-05	camisas negras sans	Medio	Completada	50
24	35	11	SUDADERAS NIKE	43	Alta	2026-07-05	SUDADERAS NIKE	Alta	Completada	43
25	35	10	SUDADERAS ADIDAS	78	Alta	2026-07-05	SUDADERAS ADIDAS	Alta	Completada	78
26	35	11	CHAQUETAS PETERIMO	144	Alta	2026-09-16	CHAQUETAS ROJAS CON BORDADOS	Alta	En Proceso	NUL
27	34	16	UNIFORMES FUTBOL	1022	Alta	2026-05-20	UNIFORMES FUTBOL NEGRO	Alta	En Proceso	NUL
28	34	16	camisas blancas	50	Baja	2026-05-20	sencillos	Medio	En Proceso	NUL
29	35	10	GORRAS NEW ERA	102	Alta	2026-10-05	ROJA CON ROSAS BORDADAS	Baja	En Proceso	NUL

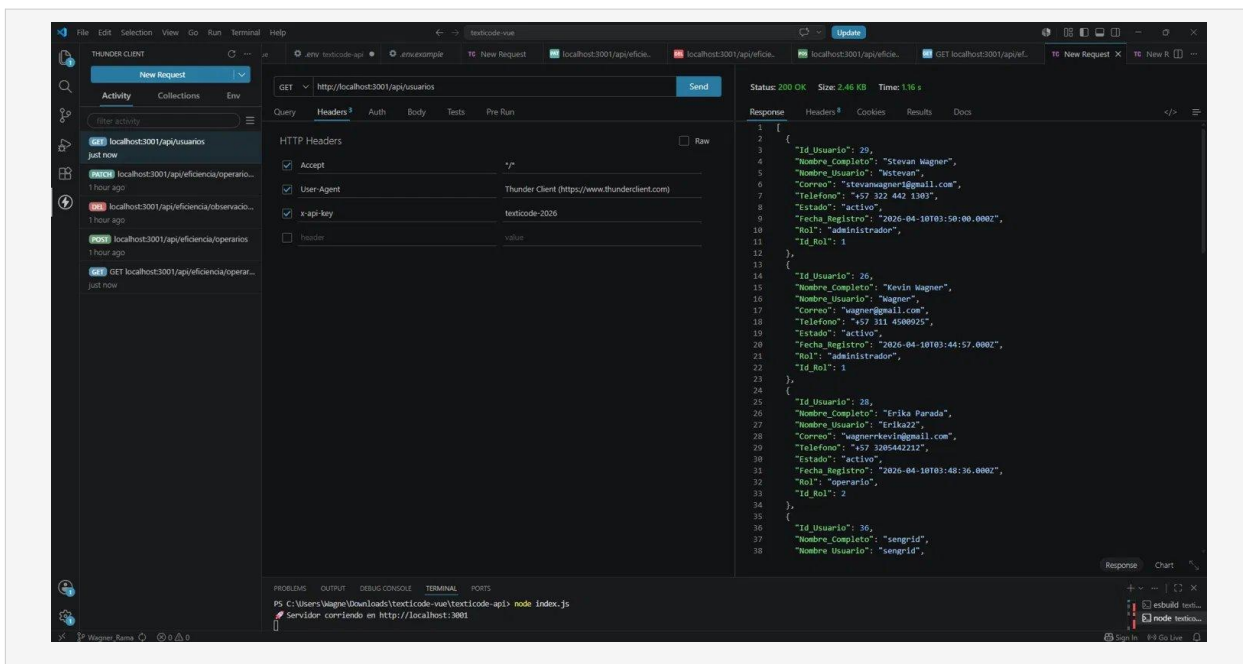
6.4 Supabase — Dashboard del proyecto texticode

Dashboard principal del proyecto texticode en Supabase mostrando: Status Healthy, Primary Database en West US (North California), CPU 2%, Disk 14%, RAM 46%, 7/60 conexiones activas y 8 Total Requests en los últimos 60 minutos (4 Database Requests + 4 Auth Requests) — evidencia de que el backend ya estaba conectado y consultando la BD.



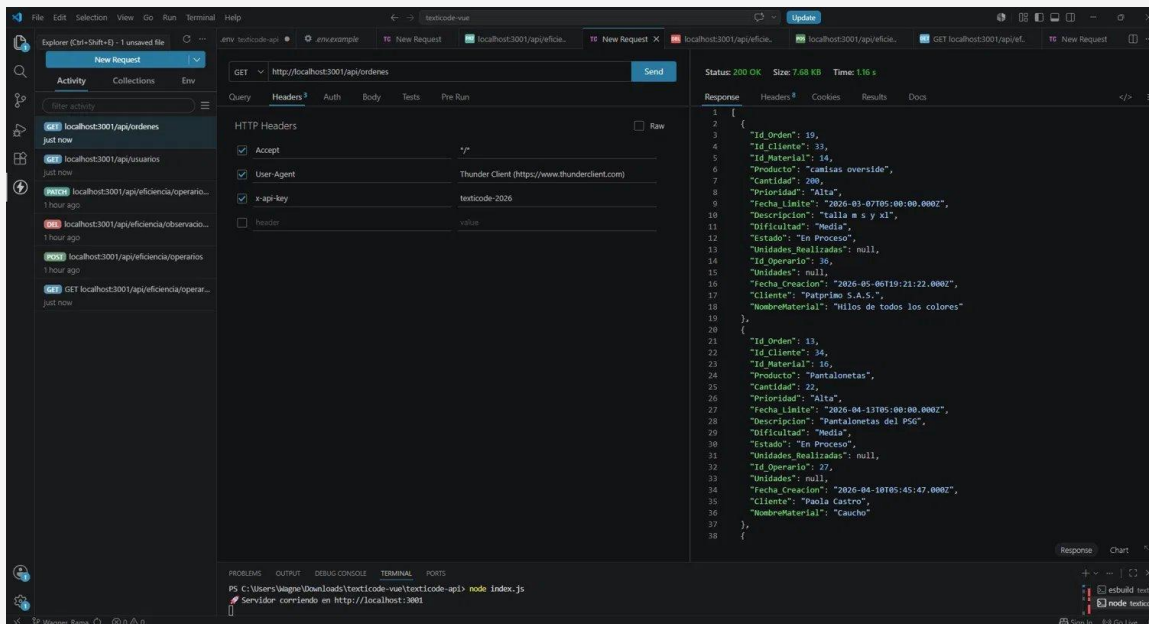
6.5 GET /api/usuarios — Lista de 11 usuarios — 200 OK

GET <http://localhost:3001/api/usuarios> con header x-api-key: texticode-2026. Thunder Client devolvió Status 200 OK, Size 2.46 KB, Time 1.16s. La respuesta incluye los usuarios del sistema con su rol JOIN: Stevan Wagner (administrador, Id_Rol: 1), Kevin Wagner (administrador), Erika Parada (operario, Id_Rol: 2), entre otros.



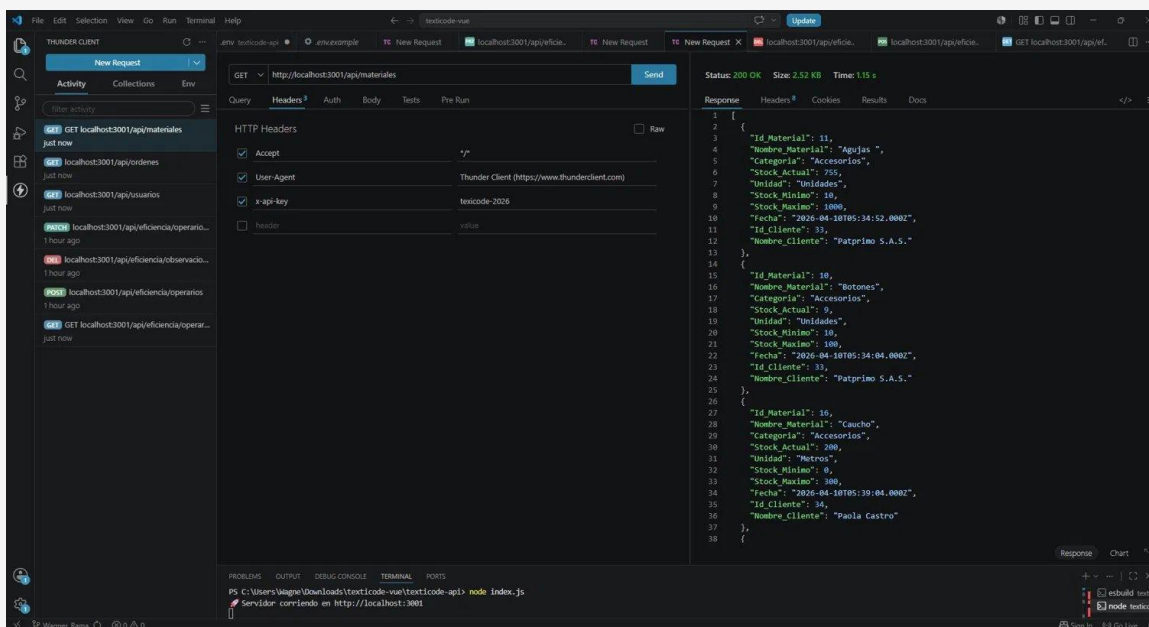
6.6 GET /api/ordenes — Lista de 20 órdenes — 200 OK

GET <http://localhost:3001/api/ordenes> devolvió Status 200 OK, Size 7.68 KB, Time 1.16s. La respuesta incluye las 20 órdenes con JOIN: Id_Orden 19 (camisas overside, Cliente: Patprimo S.A.S., NombreMaterial: Hilos de todos los colores), Id_Orden 13 (Pantalonetas, Cliente: Paola Castro, NombreMaterial: Caucho), entre otras.



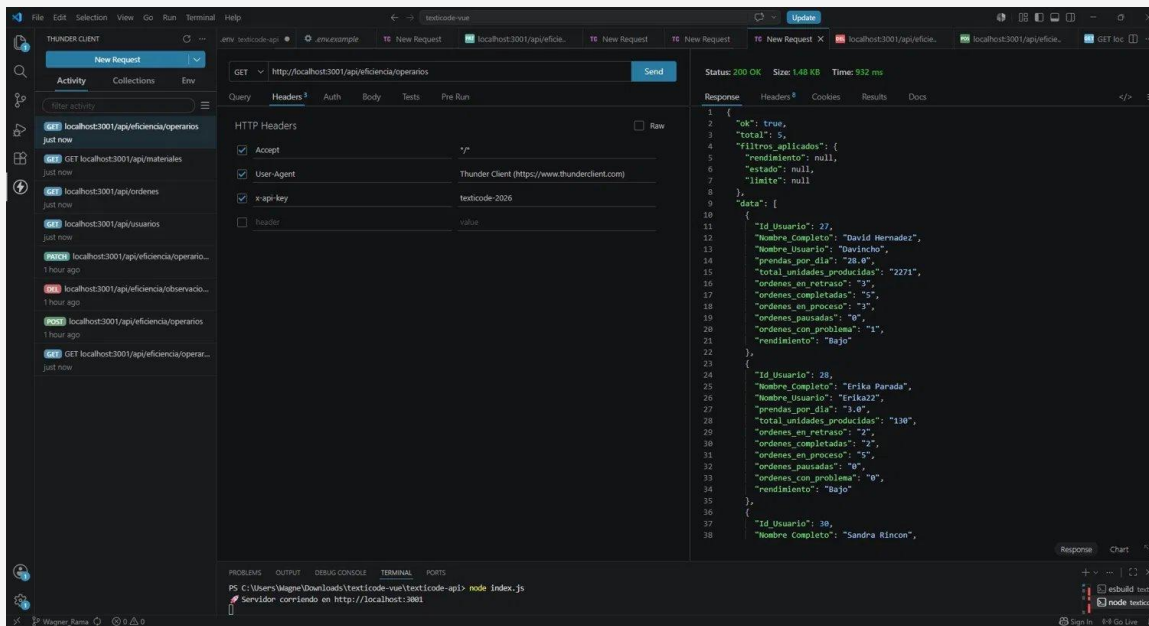
6.7 GET /api/materiales — Lista de materiales — 200 OK

GET <http://localhost:3001/api/materiales> devolvió Status 200 OK, Size 2.52 KB, Time 1.15s. Incluye materiales con cliente asignado: Aguja (Stock: 755, Accesorios, Cliente: Patprimo S.A.S.), Botones (Stock: 9, bajo stock mínimo de 10), Caucho (Stock: 200, Cliente: Paola Castro).



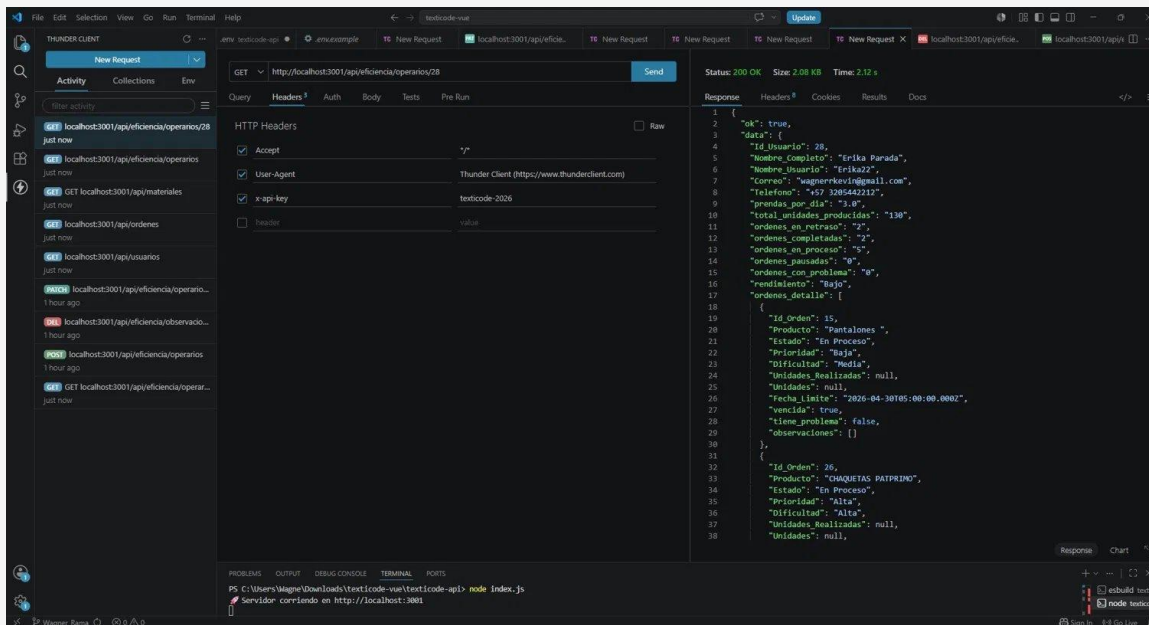
6.8 GET /api/eficiencia/operarios — Ranking con métricas — 200 OK

GET <http://localhost:3001/api/eficiencia/operarios> con x-api-key: texticode-2026. Status 200 OK, Size 1.48 KB, Time 932ms. Devolvió {ok: true, total: 5, filtros_aplicados: {rendimiento: null, estado: null, limite: null}, data: [...]}. David Hernández (Davincho): 28.0 prendas/día, 2271 unidades, 3 retrasos, rendimiento Bajo. Erika Parada (Erika22): 3.0 prendas/día, 130 unidades, 2 retrasos.



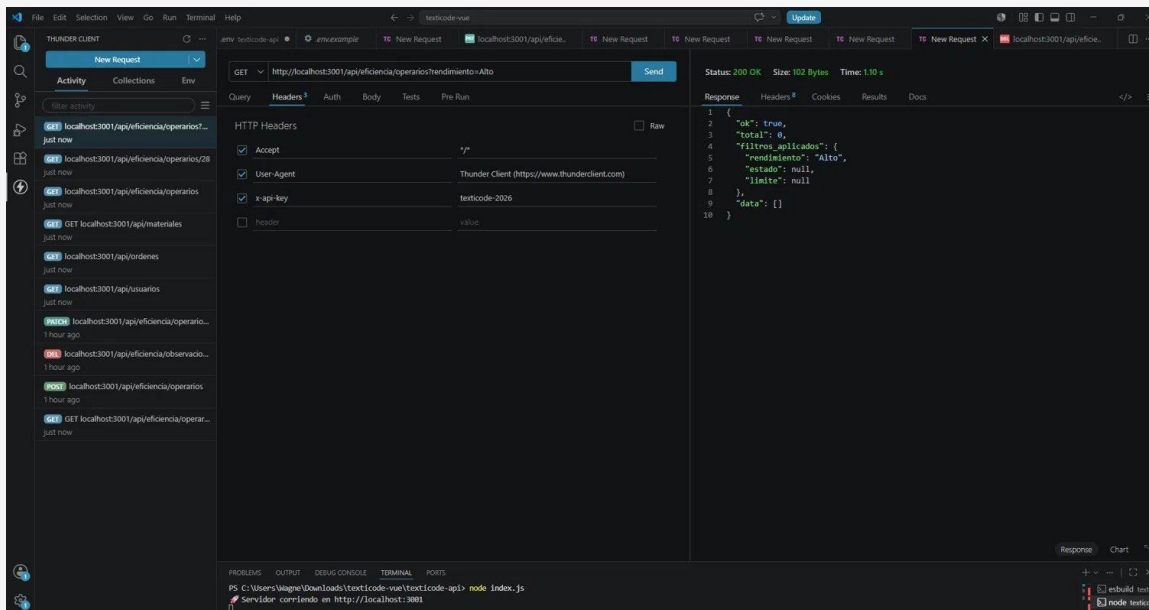
6.9 GET /api/eficiencia/operarios/28 — Path param — Perfil Erika Parada

GET `http://localhost:3001/api/eficiencia/operarios/28` (Path param `:id = 28`). Status 200 OK, Size 2.08 KB, Time 2.12s. Devolvió perfil de Erika Parada (Erika22): `prendas_por_dia: 3.0`, `total_unidades_producidas: 130`, `ordenes_en_retraso: 2`, `ordenes_completadas: 2`, `ordenes_en_proceso: 5`, `rendimiento: Bajo`. `ordenes_detalle` incluye Pantalones (Id_Orden: 15, `vencida: true`), CHAQUETAS PATPRIMO (Id_Orden: 26, `Alta prioridad`).



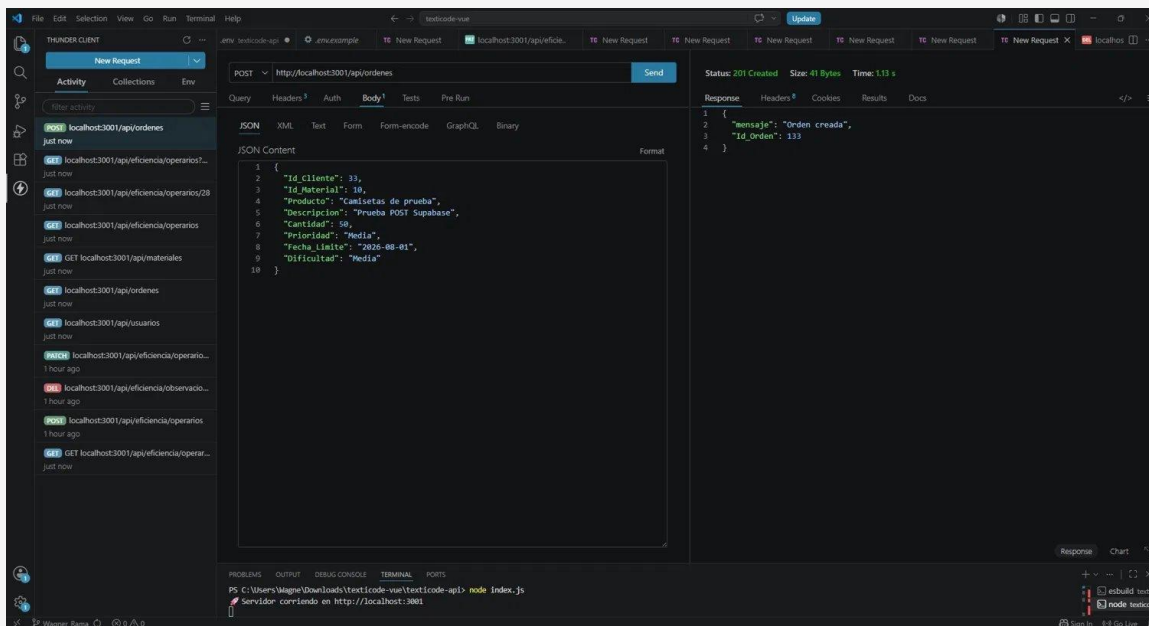
6.10 GET /api/eficiencia/operarios?rendimiento=Alto — Query param — 200 OK

GET `http://localhost:3001/api/eficiencia/operarios?rendimiento=Alto` (Query param). Status 200 OK, Size 102 Bytes, Time 1.10s. Devolvió `{ok: true, total: 0, filtros_aplicados: {rendimiento: 'Alto'}, data: []}`. Resultado esperado: ningún operario activo alcanza el umbral de rendimiento Alto (≥ 10 prendas/día) actualmente.



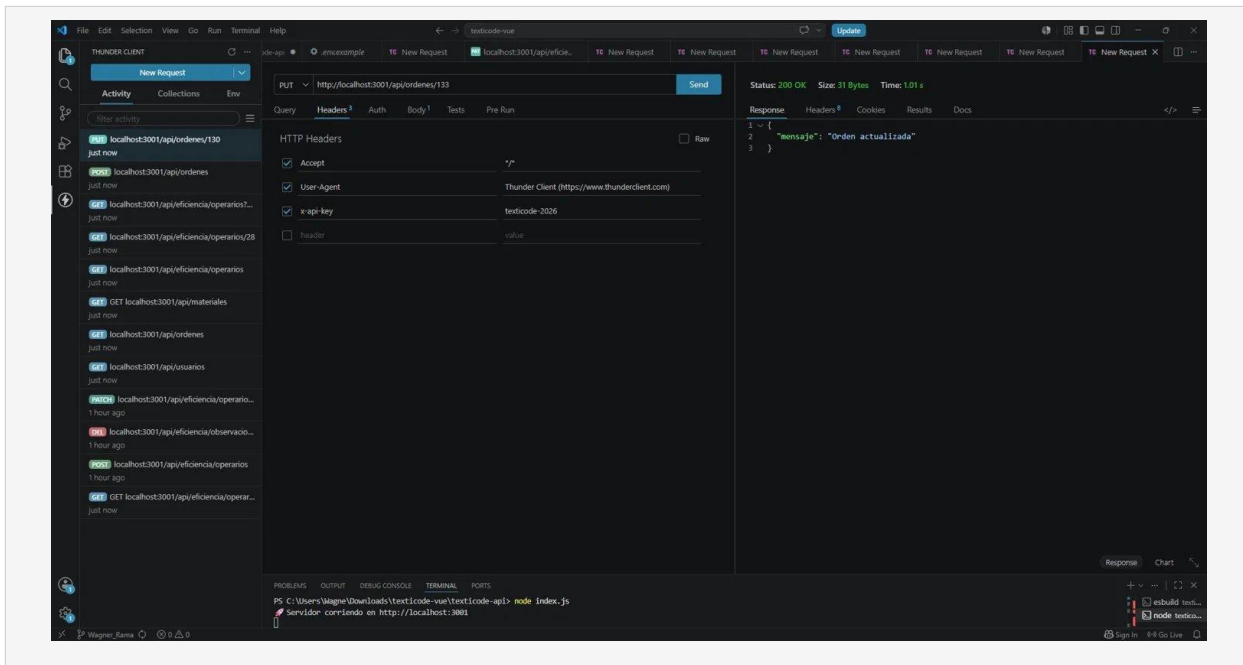
6.11 POST /api/ordenes — Crear orden — 201 Created

POST `http://localhost:3001/api/ordenes` con body JSON: `{ "Id_Cliente": 33, "Id_Material": 10, "Producto": "Camisetas de prueba", "Descripcion": "Prueba POST Supabase", "Cantidad": 50, "Prioridad": "Media", "Fecha_Limite": "2026-08-01", "Dificultad": "Media" }`. El servidor respondió Status 201 Created con `{ "mensaje": "Orden creada", "Id_Orden": 133 }`, confirmando la inserción en Supabase PostgreSQL.



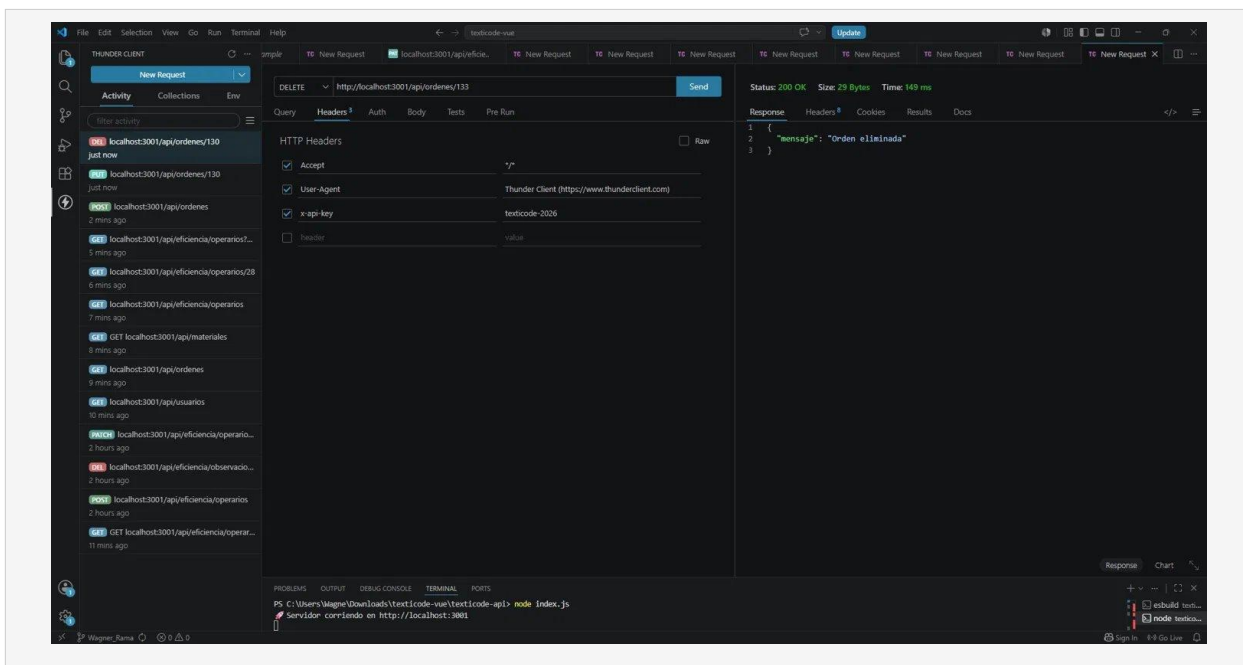
6.12 PUT /api/ordenes/133 — Actualizar orden — 200 OK

PUT `http://localhost:3001/api/ordenes/133` con body actualizando Estado a 'Completada' y Prioridad a 'Alta'. El servidor respondió Status 200 OK con `{ "mensaje": "Orden actualizada" }`, confirmando que el UPDATE se ejecutó correctamente en Supabase con `rowCount: 1`.



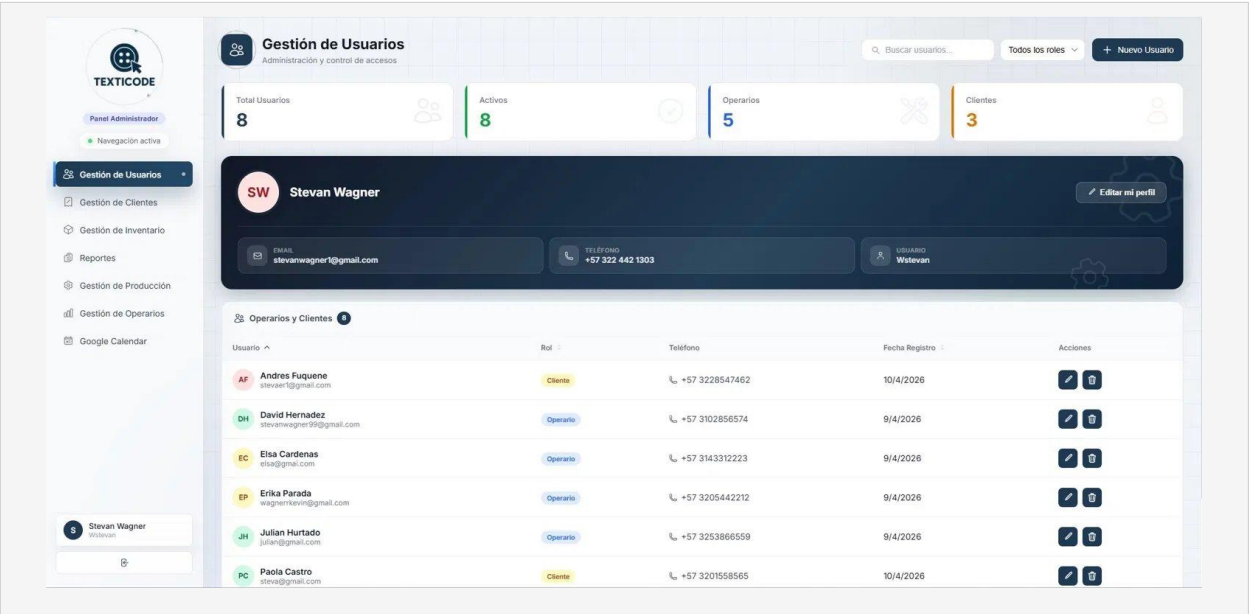
6.13 DELETE /api/ordenes/133 — Eliminar orden — 200 OK

DELETE http://localhost:3001/api/ordenes/133. El servidor respondió Status 200 OK, Size 29 Bytes, Time 149ms con {mensaje: 'Orden eliminada'}, confirmando que el DELETE se ejecutó correctamente en Supabase. La orden de prueba (Id_Orden 133) fue eliminada sin dejar registros huérfanos.



6.14 Frontend — Vista Gestión de Usuarios conectada a Supabase

Vista GestionUsuarios.vue del Panel Administrador de Texticode. Muestra los datos en tiempo real desde Supabase PostgreSQL: Total Usuarios: 8, Activos: 8, Operarios: 5, Clientes: 3. La tabla lista Andres Fuquene (Cliente), David Hernandez (Operario), Elsa Cardenas (Operario), Erika Parada (Operario), Julian Hurtado (Operario), Paola Castro (Cliente). Usuario activo: Stevan Wagner (Wstevan).



7. FLUJO DE INTEGRACIÓN COMPLETO

7.1 Diagrama de flujo — Operaciones reales probadas

#	Actor	Acción	Respuesta
1	Frontend Vue	Llama GET /api/usuarios desde GestionUsuarios.vue	Node.js recibe la petición en el router de usuarios
2	Node.js (db.js)	pool.query ejecuta SELECT con JOIN entre usuario y rol contra Supabase PostgreSQL	pg.Pool envía la query a aws-0-us-west-1.pooler.supabase.com:6543
3	Supabase	PostgreSQL ejecuta la query y devuelve los 11 usuarios del sistema con su rol	200 OK — JSON array con Id_Usuario, Nombre_Completo, Rol, Estado, etc.
4	Admin	Crea nueva orden: POST /api/ordenes con body JSON (Id_Cliente: 33, Producto: 'Camisetas...')	201 Created — {mensaje: 'Orden creada', Id_Orden: 133}
5	Node.js	INSERT INTO orden_produccion con \$1..\$10 parametrizados, RETURNING Id_Orden	Supabase asigna Id_Orden 133 via SERIAL y confirma la inserción
6	Admin	PUT /api/ordenes/133 actualiza Estado a Completada	200 OK — {mensaje: 'Orden actualizada'} — rowCount: 1
7	Admin	DELETE /api/ordenes/133 elimina la orden de prueba	200 OK — {mensaje: 'Orden eliminada'} — rowCount: 1

7.2 Justificación — API Externa Supabase vs MySQL Local

 Sobre la elección de API Externa (Supabase PostgreSQL) vs MySQL Local

Se migró de MySQL local a Supabase porque el equipo ADSO de 4 integrantes necesitaba una base de datos compartida accesible desde cualquier equipo. Con MySQL local, cada integrante tenía que mantener su propia instancia sincronizada manualmente, lo que generaba inconsistencias en los datos de desarrollo.

La migración fue transparente para el código: todos los routes del backend continuaron usando `pool.query()` con SQL parametrizado `$1`, `$2...` sin modificar ninguna lógica. Solo cambió la `DATABASE_URL` en el `.env` — de una conexión local MySQL a la URL del Session Pooler de Supabase (`aws-0-us-west-1.pooler.supabase.com:6543`).

El Dashboard de Supabase aportó monitoreo en tiempo real que MySQL local no tenía: métricas de CPU, RAM, Disk, número de conexiones activas y total de requests por minuto — información valiosa para el equipo durante el desarrollo.